

REPORT: MID-TERM PROJECT

Green Space Accessibility in Graz: A GIS-based Replication of the Craiova Study

Table of Contents

1 Original study context	1
2 Reproducibility Challenges	2
3 Methodological Adaptations.....	3
4 Annexes	4

1 Original study context

Green spaces are a vital component for the overall health of the population in cities. People are exposed to multiple pressures like air pollution, water pollution, high noise levels, thermal and visual stress. Therefore, the Accessible Natural Greenspace Standards (ANGSt) - a list of rules for the accessibility of green spaces in European cities - was established in the 1990s and revised in 2008. The standards for cities would be that nobody should live more than 300m away from a green space with at least 2 ha as well as further instructions for the accessibility of larger green spaces (Vilcea & Sosea, 2020).

The paper A GIS-based analysis of the urban green space accessibility in Craiova city, Romania by Vilcea C. and Sosea C. (2020) analysed the accessibility of public green spaces in Craiova, Romania using distance bands of 0-300m, 300-500m, 500-700m and >700m, using both Euclidian Distance and Network Analysis. The paper asks how accessible public urban green spaces are to the residents and how this accessibility varies spatially across the city and its neighbourhoods.

The urban green spaces were derived from the land cover vector data from the European Corina Land Cover datasets as well as from data published by the Urban Atlas. Green areas that were missing in these datasets were then added by

digitalization. Open Street Map was used for the street network. Finally, the census-based demographic data was provided by Romanian statistical authorities. The study uses ArcGIS for all of their analysis (Vilcea & Sosea, 2020).

The main results of the paper show that large parts of Craiova have good access to public green spaces within 300-500m, but there are still significant parts of the city without access within 700m - especially within peripheral or newly built residential areas. The network-based distances reveal lower effective accessibility than the Euclidian buffers, because the street layout and barriers lengthen the walking routes. Taking everything into consideration, the authors evaluate that targeted urban planning interventions are needed to improve green space accessibility in underserved parts of Craiova (Vilcea & Sosea, 2020).

2 Reproducibility Challenges

The first challenge was choosing the best option for the dataset depending on availability - especially considering the high-resolution population dataset. The highest resolution we could get were district-based, while the paper had higher resolution and therefore the results for the population based research approach were more detailed. The original paper uses Corine Landcover data for their public green spaces but we decided to rely mainly on Open Street Map Data. Therefore we used OSM tags and rules for our study and there could arise a reproducibility challenge: small changes in the tags for OSM filters or additional manual exclusion will change the results for public green spaces. Additionally, as Open Street Map is edited over time, repeating our workflow with our code in the future may lead to different green spaces or streets.

Another key reproducibility challenge is the limited technical detail in the original paper's methods. The study outlines the general idea (park entrances, walking network, distance bands) but not the exact algorithms or parameters for critical steps like deriving entrances, simplifying the network, or constructing accessibility zones. This forced us to choose our own implementation: automatically taking every intersection between park boundaries and streets as an entrance, inserting virtual nodes and splitting edges using distance thresholds, and building 50 m grid-based accessibility zones from interpolated network distances. These choices are clearly scripted and transparent, but they are still interpretations of the original concept rather than an exact replication. As a result, another team making reasonable but different assumptions could obtain slightly different accessibility patterns and statistics. One

main difference here is that the original paper uses ArcGIS whilst we conducted the entire workflow using Python.

Despite these challenges, the core accessibility patterns described in the original study could be reproduced at a conceptual level, particularly the systematic difference between Euclidean and network-based distances.

3 Methodological Adaptations

Whilst the original paper uses land-use classes from the CORINE land cover - so the public green spaces are defined by local authorities - we decided to use mainly Open Street Map data for this mid-term. This adaptation was implemented to keep the python script more simple for us. But therefore, there is of course a bias for the category “public green spaces” since we made our own decisions on what areas to include and which not to include. However, we tried to keep it as reproducible as possible by including a clear overview on what areas were included and which not.

Another methodological adaptation is how accessibility is summarised and visualised for Graz. The original paper presents accessibility primarily through maps and statistics for administrative units based on its GIS tools, but does not describe a fully scripted, reproducible workflow. In our project, all steps—from distance computation to aggregation and cartographic layout—are implemented in Python. This moves from a more manual GIS workflow to a programmatic one, which changes implementation details but improves transparency and makes every figure directly reproducible from the code.

Due to differences in data sources, definitions of green spaces, population resolution, and implementation details, our results for Graz are not directly comparable to those of Vilcea & Şoşea (2020). However, our workflow addresses the same accessibility questions and reproduces the important qualitative pattern that network-based walkways have lower effective accessibility than simple Euclidean buffers.

4 Annexes

References

Vilcea, C., & Şoşea, C. (2020). A GIS-based analysis of the urban green space accessibility in Craiova city, Romania. *Geografisk Tidsskrift-Danish Journal of Geography*, 120(1), 19-34.

Data

Population:

https://www.graz.at/cms/beitrag/10034466/7772565/zahlen_fakten_bevoelkerung_bezirke_wirtschaft.html

Description of Work and Responsibilities

Generally, all group members participated in this project and there were no internal conflicts regarding responsibility and participation.

Duit, Vita / Hochreiter, Tobias

Code-writing together:

- Data preparation, Data integration and Data management
- Calculations
- Preparation of the two analysis methods (Euclidean and network-based)
- Additional organisational tasks (repository management, structuring the notebook, code cleaning)

Duit, Vita: Accessibility Raster Calculation, General Greenness

Hochreiter, Tobias: Implementation of Euclidean Buffer method, Calculations with population

Erdler, Miriam

Visualization of all the results

- Report and Layout for presentation
- Code review, removal of redundant cells, etc.
- Refinement of code comments

Fleischhacker, Yvonne:

- Visualizations of plots (interim results) and Results of Method 1, KDE, Population Density & Poor Access
- Extraction of the original workflow from Vilcea & Şoşea (2020)

- Code review, removal of redundant cells, etc.

Refinement of code comments

Use of AI tools: Prompts & Reflection

It should be noted that some intermediate results generated in response to AI inputs (including exploratory plots, alternative layouts, and test routines) no longer appear in the cleaned-up version of the Jupyter Notebook. These elements were intentionally removed during the refinement process.

Erdler, Miriam:

For all AI prompts, Perplexity Pro was used, since it is provided by Uni Graz. Furthermore, the AI which is integrated in VS Code was sometimes applied for smaller code snippets like adding a North Arrow.

“I have the following code for mapping public green spaces in Graz by category. How can I change the colour of the three categories? [Include Code]”

→ This was a very quick and straight forward way to change the colour of the map, since the first code used automatic colours.

“Error Code: This gave the following error: ValueError: Invalid RGBA argument: nan [include my own code].”

Result of perplexity: “That error means at least one row in public_green_categories['category'] is NaN or not in your color_map, so .map() returns NaN which Matplotlib cannot use as a color. Use a safe mapping with a fallback color:”

→ helped me understand the error better and work through the bug.

“How can I make the legend box a bit bigger with bigger font size for this plot: [include my own code]”.

→ very fast and straight forward answer with the new code.

Taking everything into consideration, most of the time the AI prompts worked very well, and it was not necessary to proceed with follow-up questions. One aspect that I learned here was to include in the prompt for plots right in the beginning where I want cartographic elements like north arrow or scale bar by stating for example “put the north arrow in the top left corner and the scale bar in the lower left corner”.

Fleischhacker, Yvonne:

"I have a choropleth map of population density by district in Graz and a dataset of residential areas with poor access to public green spaces: >700 m walking distance. Here is the code [...], Here ist the screenshot [...]
How can I integrate these areas into the population density map?"

-> The approach worked well as an initial result; I only needed to adjust the color.

"I would like to create a kernel density map of public green spaces in Graz using Python.

Here is a screenshot of the current map layout [screenshot of the map]. I want to keep the general layout."

"Can you help me write code for that? Based on the code for the district map with the centroid polygons. Here ist he code [...]."

-> It worked reasonably well. However, the original layout was not preserved; the legend was initially oddly scaled. I therefore adjusted the colors, added a north arrow, and eventually chose a different bandwidth. I also refined the legend by requesting a classification into three categories.

"Please help me resolve this merge conflict"

"Please helb me to restore the initial version of the project."

-> The solution worked very well.

"Can you help with the code: I want to creat a street network map of Graz with visble nodes"

-> It worked very easily, and the results looked great in different colors, which I then adjusted.

"I want to highlight the main roads in the street network map while keeping minor roads visually subdued.

-> The solution worked very well. I check which tags were used: OSM classifies streets using the "highway" tag, which distinguishes major road types.

Overall, working with AI tools such as ChatGPT and GitHub Copilot integrated into VS Code was very helpful throughout the project for me. ChatGPT works relatively well, especially when used for step-by-step instructions, explaining code snippets, or interpreting errors. However, it is still necessary to carefully review the output, as the

model sometimes suggests solutions that are more complicated than necessary or introduce unnecessary complexity.

ChatGPT was particularly helpful when I needed explanations for more complex processes or when tasks had to be broken down into smaller, manageable steps. Many results had to be refined iteratively: if the prompts asked for “too much” at once, elements were often missing from the output. Incremental work yielded the best results.

What I would improve next time:

In future projects, I would structure the use of AI more consciously. This includes writing clearer instructions, separating conceptual questions from code generation tasks, and validating intermediate steps more consistently before continuing. I would also keep more systematic documentation. This also has a better learning effect for me.

Hochreiter, Tobias and Duit, Vita:

Perplexity:

“I want to do an analysis of green space accessibility and also an analysis of how Green a district / area is. so for the accessibility, i need all public parks and urban forests, and for how green the district is, i would also consider private gardens, meadows, grass, etc. can you write me a query to load all of these features?”

→ worked very well instantly, working query to load features

“what about graveyards and alleyways? are they relevant for the green area analysis?”

→ gave us arguments for keeping graveyards and leaving alleyways out, we acted accordingly

Claude:

"I have a gdf with green spaces in Graz dissolved into 1 multipolygon and a street network of Graz. I want to find access points of the green spaces and was thinking of intersecting the green spaces with the edges of the network to get these? how to do this?"

→ quite a lot of debugging but worked well

Claude:

"hello! I have a network graph of the city of Graz and the access points to green areas (intersections between edges of the network and green areas). Now I want to do one to all routing to the access points to make buffers of walking distances of different distance steps. Now I want to know how I can do a routing, because the access points are not really in the network... so I can't directly do the routing. I was thinking of finding nearest nodes in the network that are closest to the access points. But then the nearest node might not be representative to the real access point, because it might find a node behind a fence or something as the nearest node. Can you think of any option how I can do a routing to the access points?"

→ more complicated task but worked well with some debugging and more questions about details in the code

Claude for accessibility map (after deciding buffers are not suitable):

Maybe buffers are not the best choice. Is there another way to separate the area into parts that are within walking distance of 100, 200 and 300m? Maybe by checking the value of `access_dist` for most of the area? So I could make a map that shows areas that are within these distances and those which are not and also to have some areal statistics results.

→ quite a lot of debugging and asking follow-up questions necessary, but good result in the end that I wouldn't have come up with without the input Claude for "general green-ness":

"Now I also want to do an analysis of general greenness in the districts of Graz. I have `green_spaces` from `osmnx`, which includes all green spaces in Graz, and I have

`gdf_residential` also from `osmnx` and `gdf_districts` with the districts of `graz`.”

can you write me a short script with the following outputs:

- * percentage of green area (`green_spaces`) in the whole of `graz` (`gdf_graz`)
- * percentage of green area in each district

→ worked very well, no debugging necessary, made it a lot easier and faster than typing everything by ourselves

Perplexity:

“now i want to calculate deficient areas with my output `zones_gdf` like i did it here:

[code deficient areas euclidean distances]

how to do this with my output `zones_gdf`?”

→ worked instantly

Generally, we used the AI mainly for code-writing, but also for generating conceptual ideas. Even though the code generated by AI often didn't work on first try, it could be debugged (often also using AI and explaining error messages with AI). Using AI allowed us to implement more difficult tasks than we otherwise would have been able to but also made simple tasks quicker.

We also used the integrated AI functions of VS Code. This worked great for simple tasks, but for more complex steps, the AI was more of a good starting point than a complete solution. If the comments in the code were written accurately and clearly, the integrated AI was able to generate working code directly from these comments. Many of the comments in the notebook can therefore be understood as prompts.